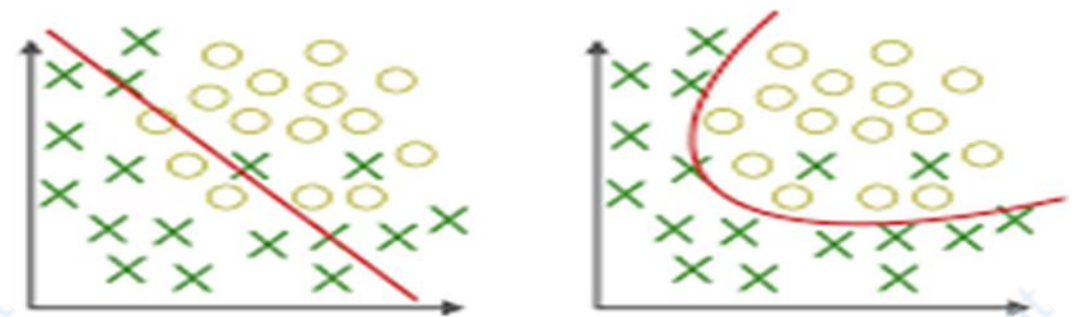
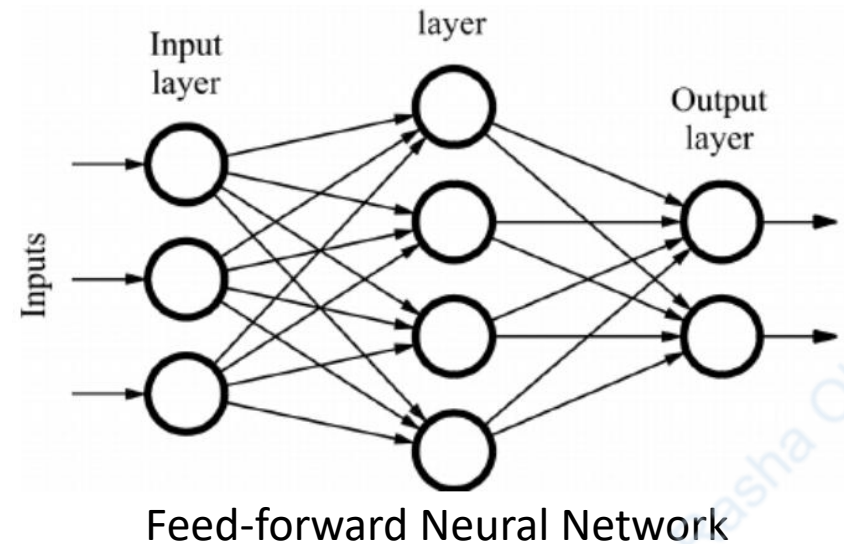


Feed-Forward Neural Networks

Rasha Obeidat, PhD

Introduction

- **Feed-Forward Neural Networks (NNs)**
Also known as multi-level perceptron (MLP).
- Design as based on an analogy with the structure of the brain.
- A family of classifiers with non-linear decision boundary.



Linear vs. Non-linear decision boundaries

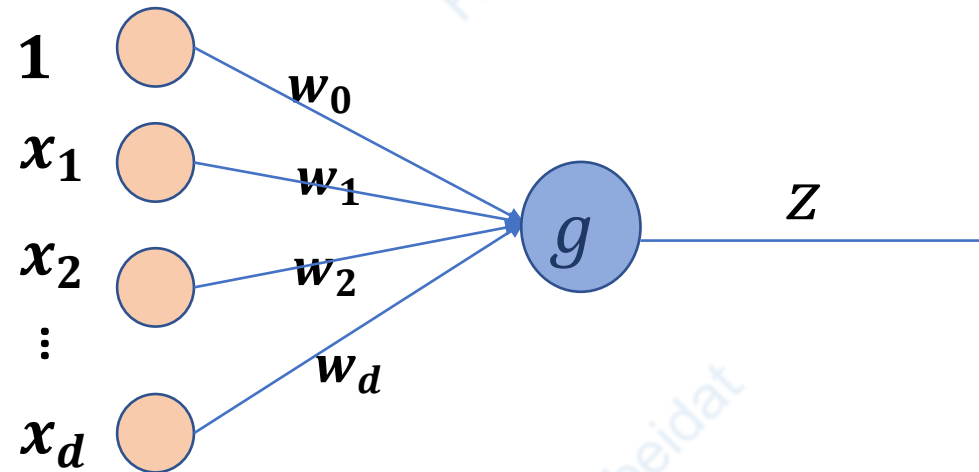
A Single-Layer Neural Network

- Given a training data: $\{(x^1, y^1)\}_{i=1}^n$, $x^i = ([1 \ x_1 \ x_2 \ \dots \ x_d])^T$, single-layer neural network is a combination of linear layer and non-linear **activation function** g .

$$z = g(w^T x)$$

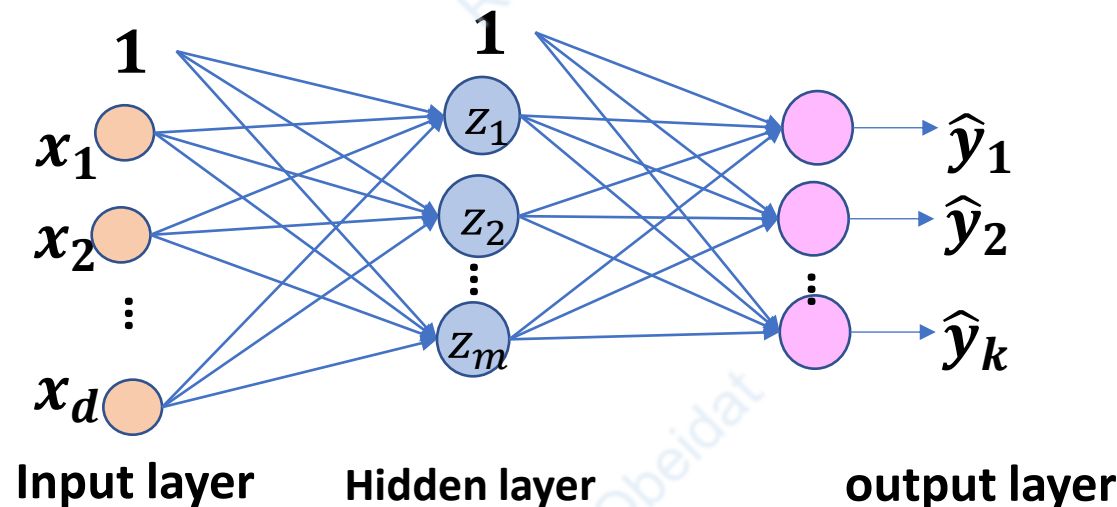
The z is used to compute another function (e.g., decide $\hat{y}$)

- Logistic regression as an example.

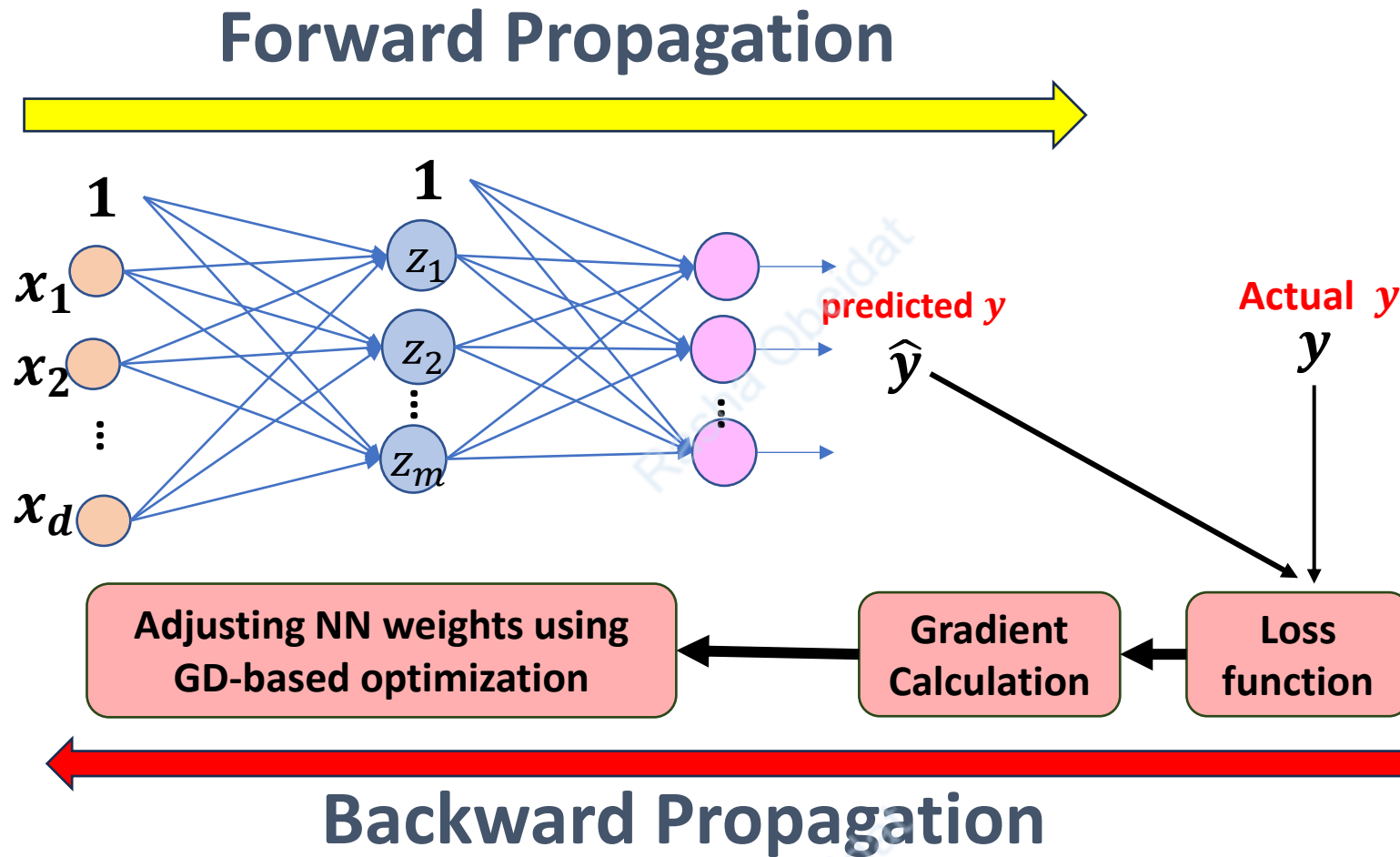


Feed-forward Neural Network (A Multi-Layer)

- It is a set of neurons, connected by weighted connections.
- Consists of an input layer, output layer and one or more hidden layers.
- Each layer receives its inputs from the previous layer and forwards its outputs to the next.
- The depth of the network equal the number of layers of trained weights.



Forward Propagation and Backpropagation in NNs



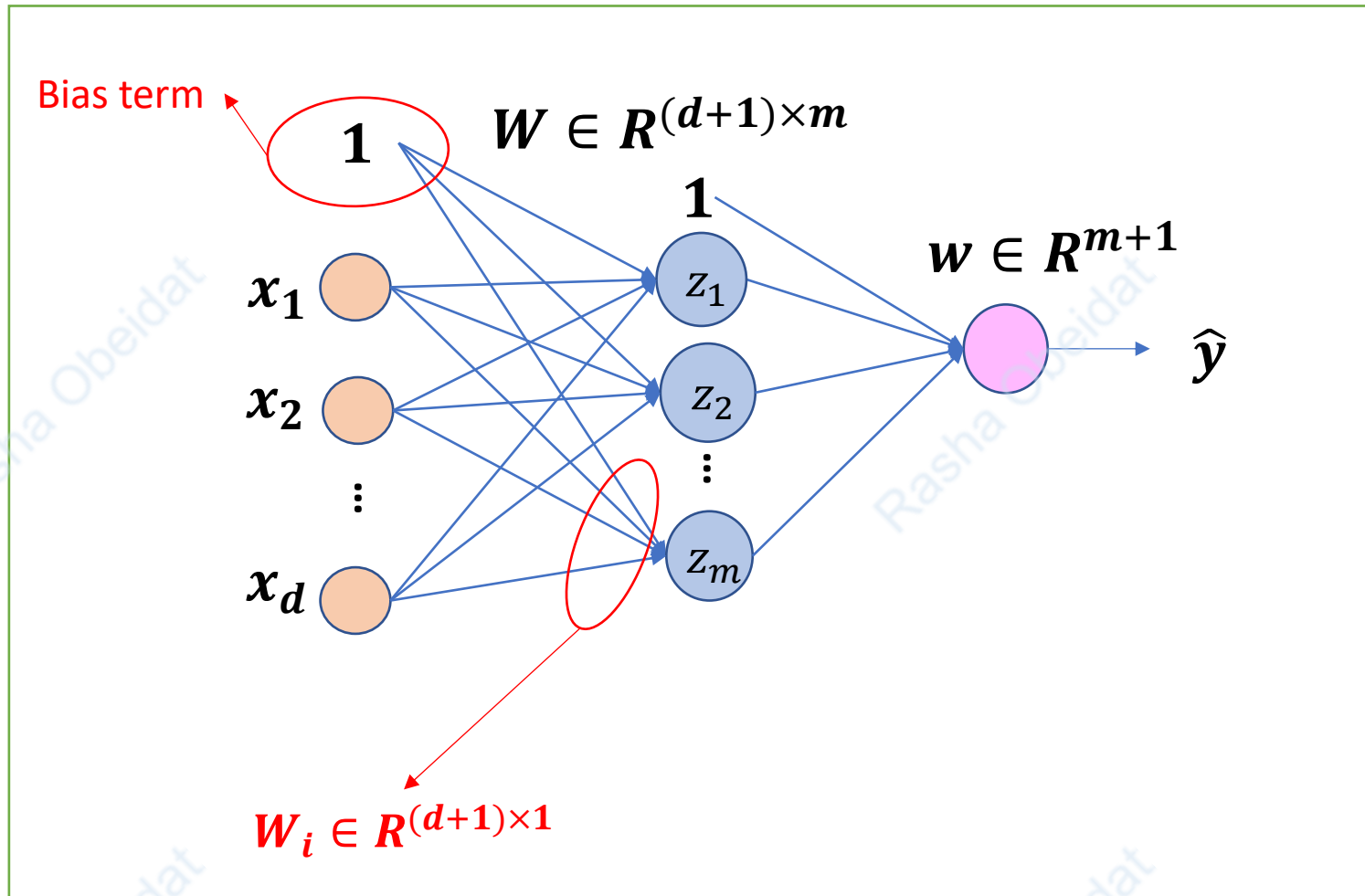
Feed-forward Neural Network

- **Input layer:** receive inputs and pass them on to the next hidden layer. The number of neurons in the input layer equals the number of dimension in the data representation (number of features)
- **Output layer:** The output layer is the predicted value (e.g., the probability of predicting each class in classification problem).
- **Hidden layer:** which are neuron nodes stacked in between the *input* and *output*, layer allowing neural networks to learn more complicated features.

Forward Propagation

- **Purpose:** Computes the output of the neural network ($\hat{y}$).
- **Process:**
 - **Flow of Computation:** Moves from the input layer to the output layer.
 - **Direction:** Each layer's output serves as the input to the next layer, progressing sequentially from the first layer to the last.
 - **Calculations:**
 - Multiply inputs from the previous layer by the corresponding weights.
 - Add biases.
 - Apply an activation function to produce the output for each neuron.
 - **Final output (classification) layer** produces the predicted label or value ($\hat{y}$) after processing through all the network layers

Forward Propagation



Binary NN with one hidden layer

$$\begin{aligned} a &= W^T x \\ z &= g(a) \\ \hat{y} &= h(w^T z) \end{aligned}$$

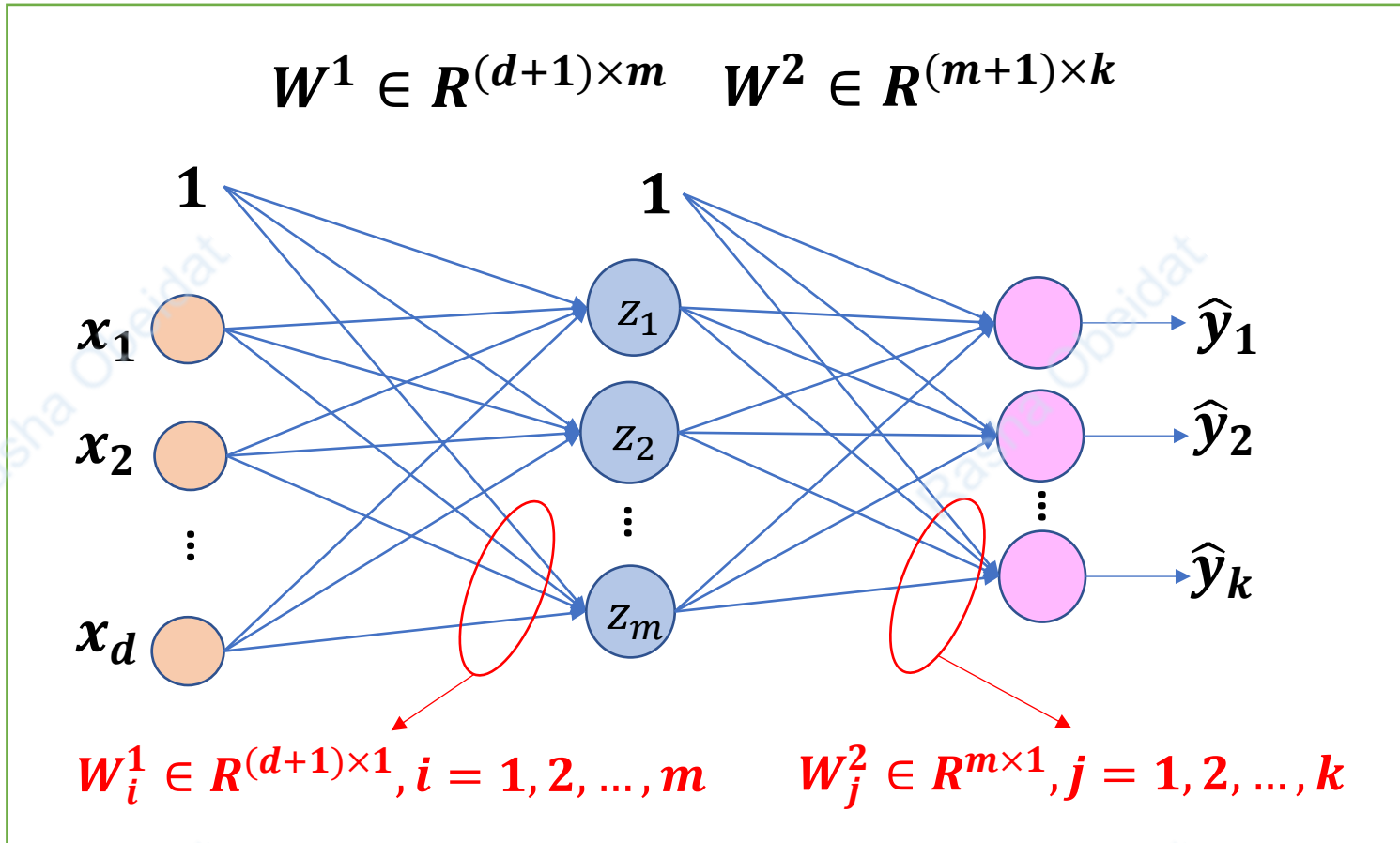
$$\begin{aligned} a_i &= W_i^T x \\ z_i &= g(a_i) \end{aligned}$$

$$W = \begin{bmatrix} [W_1] & [W_2] & \dots & [W_m] \end{bmatrix}^T$$

W_i is the i^{th} column of W

$h(x)$ and $g(x)$ are activation functions (e.g., σ).

Forward Propagation



$$a = W^1{}^T x$$

$$z = g(a)$$

$$\hat{y} = h(W^2{}^T z)$$

$$a_i = W_i^1{}^T x$$

$$z_i = g(a_i)$$

$$\hat{y}_j = h(W_j^2{}^T z)$$

$$W^1 = \begin{bmatrix} [W_1^1] & [W_2^1] & \dots & [W_m^1] \end{bmatrix}^T$$

$$W^2 = \begin{bmatrix} [W_1^2] & [W_2^2] & \dots & [W_k^2] \end{bmatrix}^T$$

W_i^1 is the i^{th} column of W^1

W_j^2 is the j^{th} column of W^2

Multi-class ANN with one hidden layer

$h(x)$ and $g(x)$ are activation functions (e.g. g can be σ , and h can be softmax).

Back-propagation

- **Purpose:** Adjusts weights to minimize the error between the predicted output $\hat{y}$ the actual (*Ground truth*) label (y).
- **Process:**
 - **Error Calculation:**
 - The loss function computes the error at the output layer.
 - **Gradient Computation:**
 - Uses the chain rule to calculate gradients of the loss with respect to each weight in the network.
 - Propagates the error backward through the layers.
 - **Weight Update:**
 - Adjusts weights incrementally using the computed gradients to reduce error.
 - **Direction:** Traverses the network in reverse order, from the output layer to the input layer

Backpropagation and Chain rule

- The derivative measures the sensitivity to change of output value with respect to a change in its inputs.

- Example 1:

$$b = 3a :$$

- if $a = 4 \rightarrow b = 12$
- if $a = 4.001 \rightarrow b = 12.003$
- This means that when we increase a by 0.001 , then b goes up three times as much a increases. Mathematically we say $\frac{\partial b}{\partial a} = 3$.

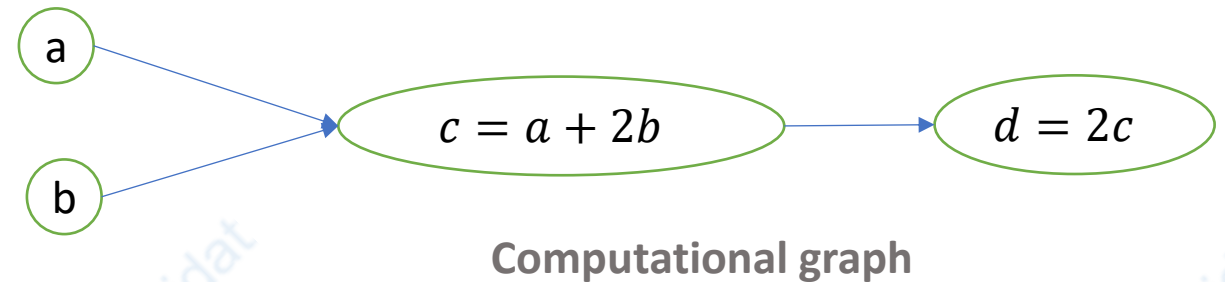


Computational graph

Backpropagation and Chain rule

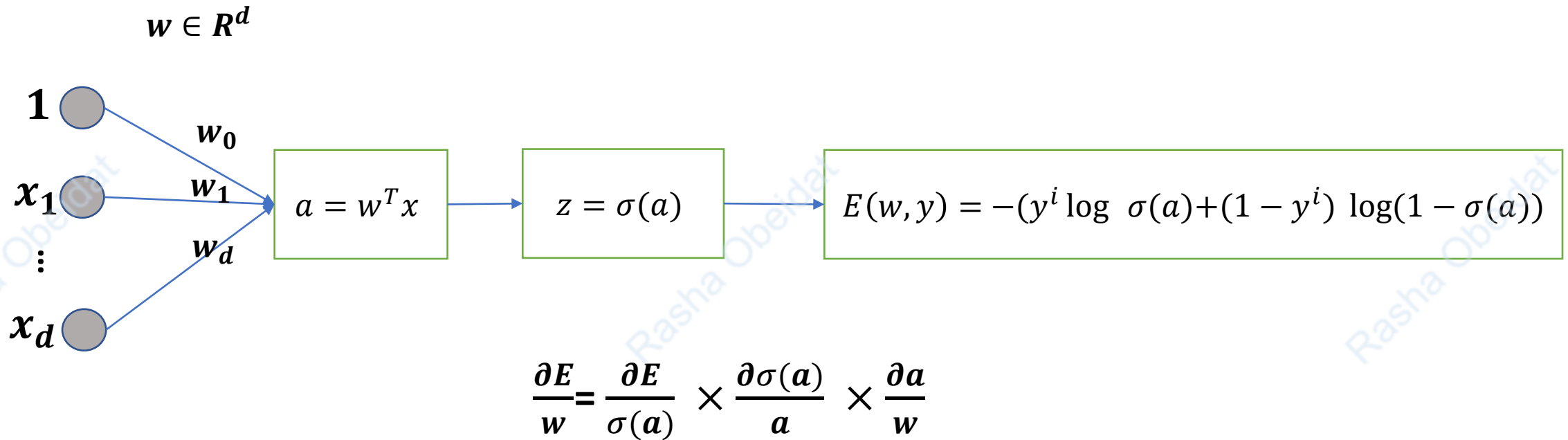
- Example 2:

$$c = a + 2b, d = 2c$$

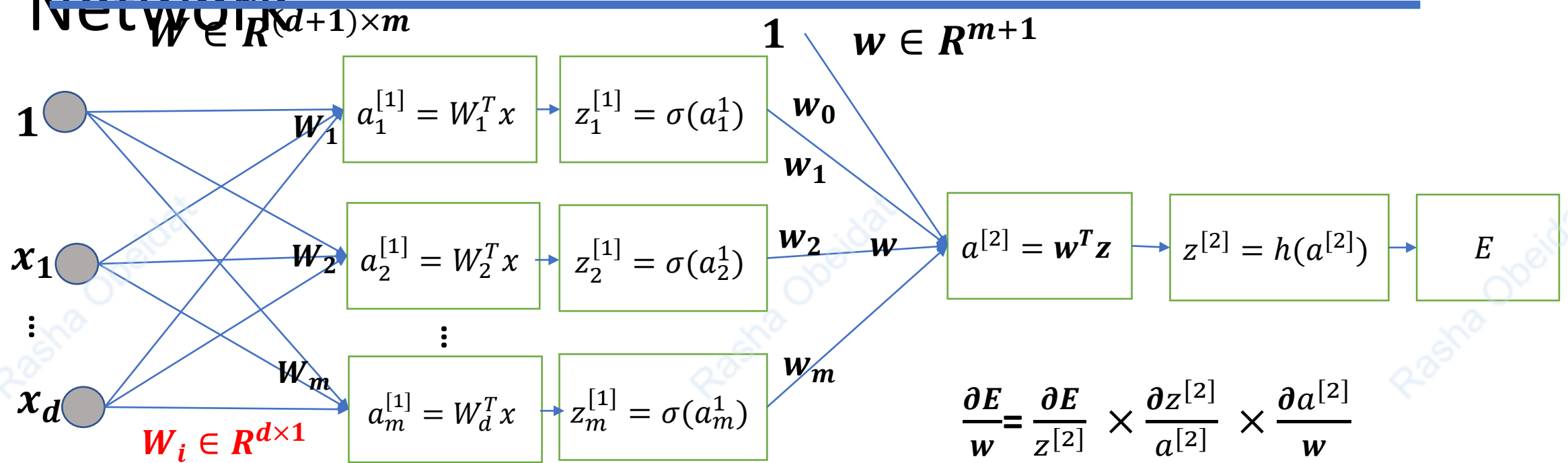


- if $a = 4, b = 2 \rightarrow c = 8, d = 16$
- if $a = 4.001, b = 2 \rightarrow c = 8.001, d = 16.002$
- if $a = 4, b = 2.001 \rightarrow c = 8.002, d = 16.004$
- When we increase a by 0.001, then c goes up as much a increases, and d goes up 2 times as much c increases, this means that d goes up 2 times as much a increases. Mathematically we say $\frac{\partial d}{\partial a} = \frac{\partial d}{\partial c} \times \frac{\partial c}{\partial a} = 1 \times 2$ **(chain rule)**.
- Likewise, when we increase b by 0.001, then c goes up 2 times as much b increases, and d goes up 2 times as much c increases, this means that d goes up 4 times as much b increases. Mathematically we say $\frac{\partial d}{\partial b} = \frac{\partial d}{\partial c} \times \frac{\partial c}{\partial b} = 2 \times 2$

Derivatives of logistic Regression



Derivatives of 2-level feed-forward Neural Network



$$W = \begin{bmatrix} [W_1] & [W_2] & \dots & [W_m] \end{bmatrix}^T$$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial w}$$

$$\frac{\partial E}{\partial W} = \frac{\partial E}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[1]}} \times \frac{\partial z^{[1]}}{\partial a^{[1]}} \times \frac{\partial a^{[1]}}{\partial W}$$

$$\frac{\partial E}{\partial W_i} = \frac{\partial E}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z_i^{[1]}} \times \frac{\partial z_i^{[1]}}{\partial a_i^{[1]}} \times \frac{\partial a_i^{[1]}}{\partial W_i}$$

The Backpropagation Algorithm

Mini-batch gradient decent for a binary 2-levels feedforward NN

Input: Data (X, Y) divided into B batches $\{X_b, Y_b\}_{b=1}^B$ according to some batch size bs , $X_b \in \mathbf{R}^{bs \times d}$ is the matrix of feature representation of the samples in the batch b where d is the dimensionality of that feature representation and Y_b is a vector of ground truth classes of the samples in batch b .

Learnable Parameters: input-to-hidden weight matrix W and hidden-to-output weight w .

For epoch in #Epochs:

(Alternative stop criteria can be used)

for b in batches:

forward Pass: compute $\hat{Y}_b$, which's a vector of predictions for $\forall x \in X_b$

compute the error from $\hat{Y}_b$ and Y_b

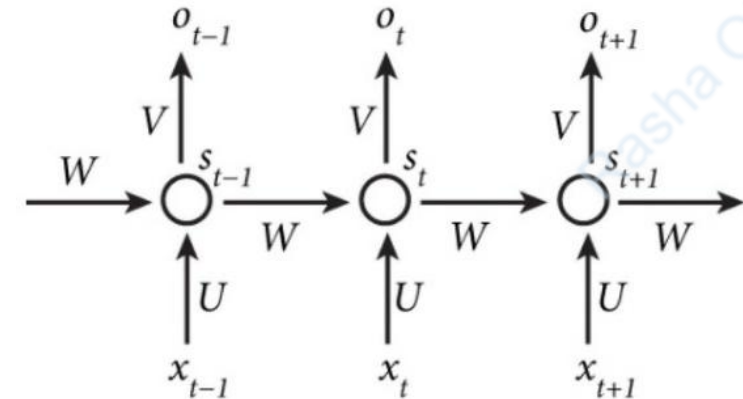
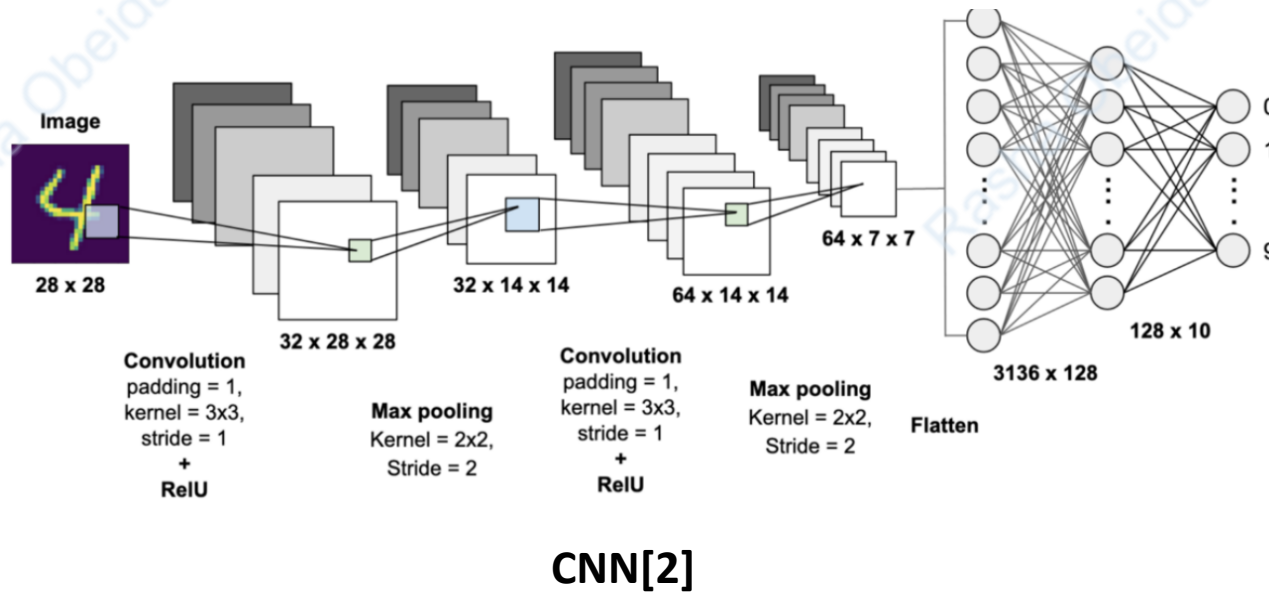
compute the gradients for the $\frac{\partial E}{w}$, and $\frac{\partial E}{W}$

Backpropagation Pass: back-propagating the error by taking gradients steps.

$$W \leftarrow W - \eta \frac{\partial E}{W}, \quad w \leftarrow w - \eta \frac{\partial E}{w}$$

Different DAG structures

- The backpropagation algorithm would work for any Direct Acyclic Graph(DAGs).



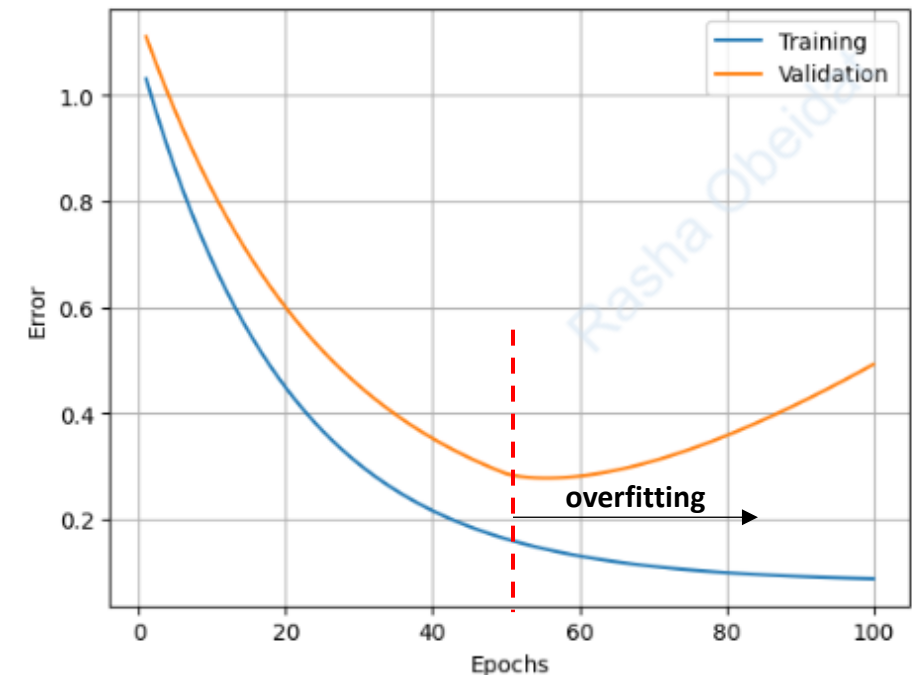
RNN

Practical Tips

- To avoid local minima problems, can run several trials starting from different initial random weights and:
 - **Best Result Selection:** Take the result with the best training or validation performance.
 - Build a committee of networks that vote during testing, possibly weighting (*ensembling*)

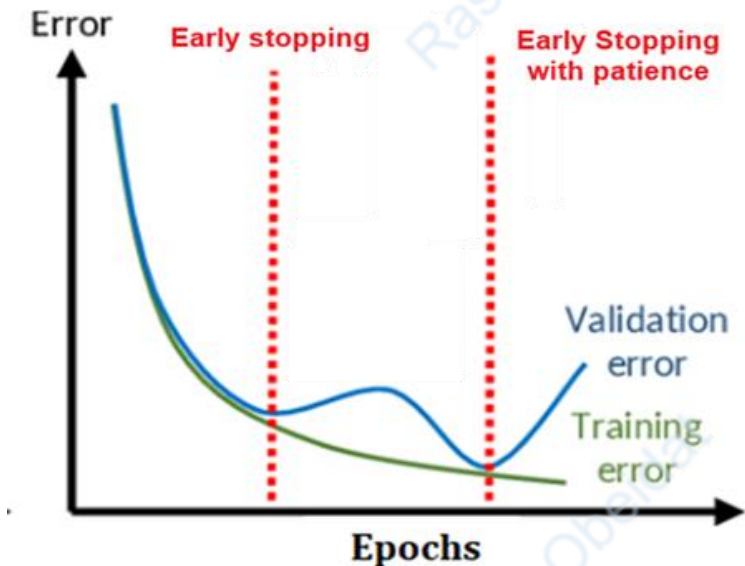
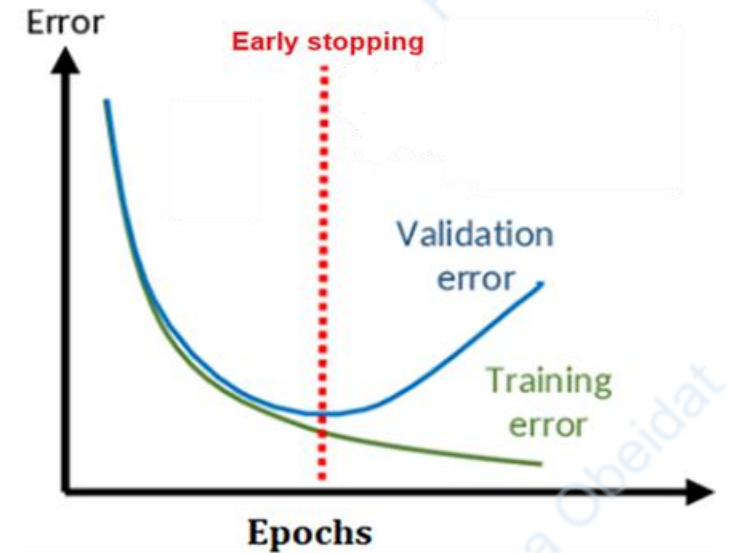
Practical Tips: Early stopping

- **Early stopping** is a *regularization* technique used to prevent overfitting during training.
 - The model's performance (e.g., loss, error , accuracy) is monitored **on a validation set** during training.
 - If validation performance stops improving (*e.g., validation loss increases or validation accuracy decreases*) training is stopped early.
- This prevents the model from
 - fitting too closely to the training data, which protects its ability to generalize to new, unseen data.
 - over-learning noise in the training data.
- **Keep in mind:** Stop training at the point of best validation performance, not at the point of best training performance.



Early stopping with patience

- **Patience** is the number of additional epochs *after the first degradation* in the monitored performance metric (e.g., validation loss or accuracy) during which training is allowed to continue.
- If no improvement occurs within these additional epochs (getting **same performance or degradation**), the training process is stopped.
- it allows for fluctuations in performance metrics during training, preventing premature termination and ensuring that the model does not stop too early, which could lead to suboptimal performance.
- Patience is a **hyper-parameter**.



Early stopping with patience

How It Works:

- **Monitoring Improvement:** As training progresses, you keep track of the validation performance metric.
- **Patience Count:** If the model's performance degrades for the specified patience (e.g., 3 consecutive epochs without surpassing the best performance observed), you stop training. During this patience counting, if an improvement is encountered, the **patience counter (not patience) resets to 0**.
- **Selecting the Best Model:** You should select the model weights from the epoch where the best validation performance was observed before the patience threshold was reached.
- Q: what is the epoch number that corresponds to the best models when we use early stopping with patience= **0, 1, 2**?

Epoch	Dev Accuracy	train Accuracy
1	0.6	0.6523
2	0.67	0.7652
3	0.72	0.7541
4	0.81	0.8456
5	0.88	0.8955
6	0.87	0.9025
7	0.94	0.9231
8	0.91	0.94563
9	0.92	0.9541
10	0.93	0.9598
11	0.92	0.9666
12	0.91	0.9778
13	0.93	0.9812
14	0.92	0.9889
15	0.92	0.9901
16	0.90	0.9932
17	0.89	0.9941

Practical Tips

- No guarantee for NN to converge, may oscillate or reach a local minima.
- Many epochs may be needed for adequate training . ***Epoch* is going through the training data once.**
- Termination criteria can be:
 - Fixed number of epochs (e.g., 100 epoch)
 - Threshold on training set error (e.g., stop when the error $< 6\%$ which is equivalent to Accuracy $> 94\%$)
 - Early stopping: increased error (or loss) on a validation set.

Practical Tips

- **Notes on Proper Initialization**

- Start in the “linear” regions: ***keep all weights near zero***, so that all sigmoid units are in their linear regions.– This will also avoid having very small gradient.
- **Break symmetry**: ensure that each hidden unit has different input weights so that the hidden units move in different directions.
 - Q. If all weights are initialized to the same value, what would happen?
 - A. every neuron in the layer will compute the same output for any given input. This leads to a lack of diversity in learning, as all neurons will update in a synchronized manner during training, learning the same features.

Loss Functions

- **Regression:**

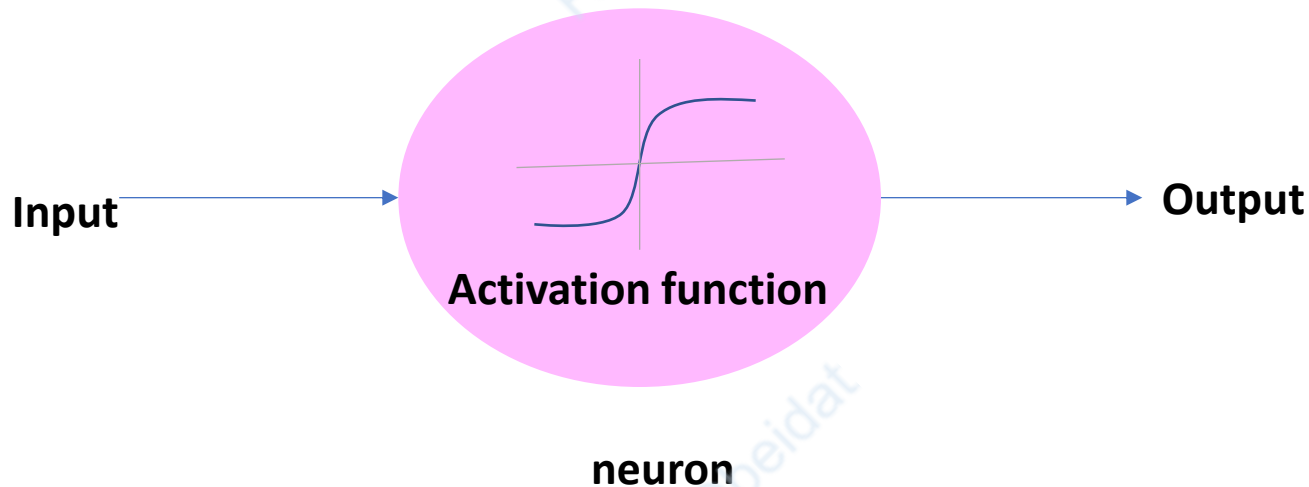
- Mean Squared Error (MAE) : $E(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$
- Mean Absolute Error (MAE) (aka L1 loss) : $\frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$

- **Classification**

- Max Margin: $E(w) = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i f(x_i; w))$
- Cross-entropy: $E(w) = -\frac{1}{n} \sum_{i=1}^n (y_i \log \sigma(x) - (1 - y_i) \log(1 - \sigma(x)))$
- Multinomial Cross-entropy: $E(w) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$

Activation functions

- Also known as *Transfer Functions* or *link functions*.
- Activation function decides whether a neuron is “fired” (“activated”) or not.
- It is important for an Artificial Neural Network (ANN) to learn non-linear complex functional mappings between the inputs and output.



Activation functions

Should be:

- **Differentiable** (or has sub derivatives): need for ***backpropagation optimization***.
- **Non-linear** : Why?
 - Allow “stacking” of multiple layers of neurons to create a deep neural network, that without nonlinearity, multiple layers ANN are equivalent to a single layer neural network, because a linear combination of linear functions is still a linear function. Besides,
 - Allow backpropagation because they have a derivative function which is **related to the inputs**, so the weights are learned based on which feature are more important.

Activation function

- Sign function: $f(x) = \begin{cases} 1 & , x > 0 \\ -1 & , x \leq 0 \end{cases}$
- Sigmoid function (Softmax for multi-class):

$$\sigma(x) = \frac{1}{(1 + e^{-x})}$$

- Hyperbolic tangent (Tanh):

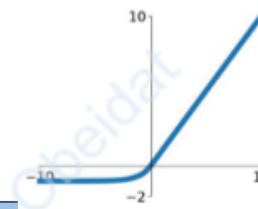
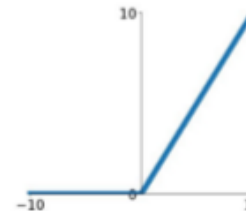
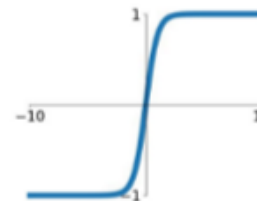
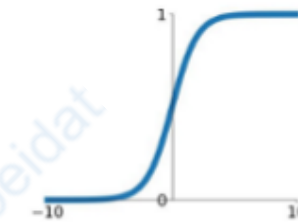
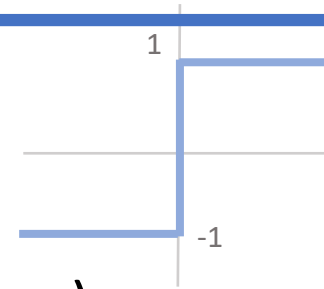
$$\tanh(x) = \frac{(1 - e^{-2x})}{(1 + e^{-2x})}$$

- Rectified Linear Unit (ReLU):

$$f(x) = \max(0, x)$$

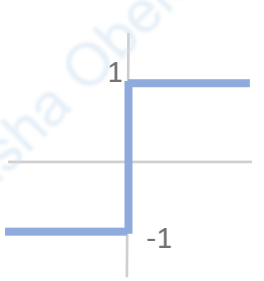
- Leaky ReLU:

$$f(x) = \max(0.1x, x)$$



Note: x here is the input to the activation function, not the input to the classifier. For example, it corresponds to $W^T x$ in the hidden layer of the feed-forward Neural Network as explained in the slide titled "[Forward-Propagation Pass]" (presented earlier).

Activation Function: Sign function



Sign function: A threshold-based activation function that turns the neuron output on and off.

$$f(x) = \begin{cases} 1 & , x > 0 \\ -1 & , x \leq 0 \end{cases}$$

Derivative of Sign function :

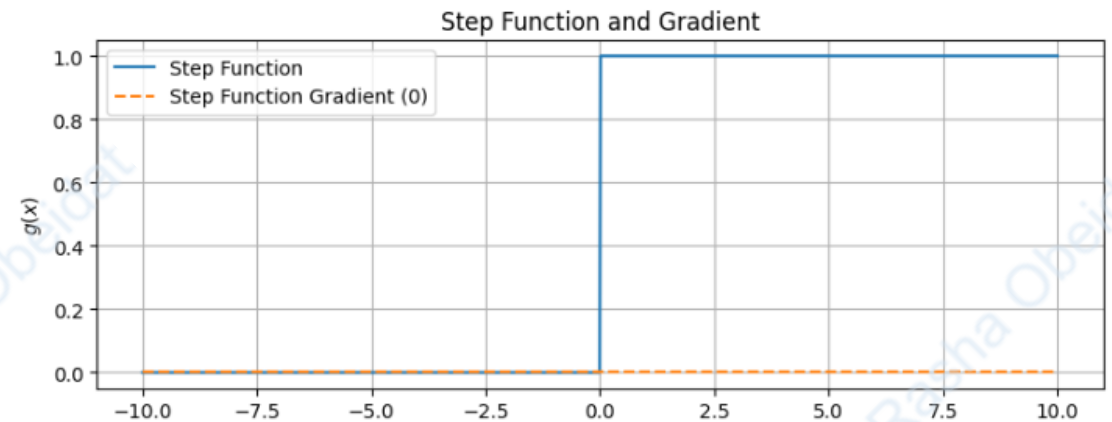
$$\frac{\partial f(x)}{\partial x} = \begin{cases} 0 & , x \neq 0 \\ \text{undefined} & , x = 0 \end{cases}$$

Pros:

- Simple and cheap to compute.

cons

- Non-differentiable .
- Discontinuously jump from -1 to 1 which is not appropriate for neural network that learn little by little. Small changes in weights move us along this curve, at some point a small change will cause a sudden change in the activation of the neuron • This will cause chaotic behavior in the network

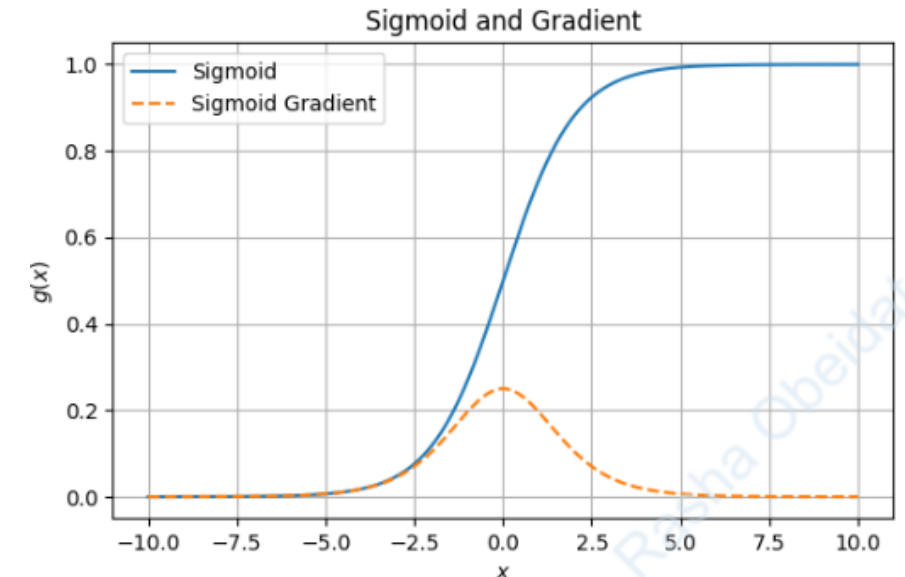


We need a continuous activation function so that the network changes smoothly when modifying the parameters

Activation Functions: Sigmoid

- Sigmoid function : $\sigma(x) = \frac{1}{(1+e^{-x})}$
- Derivative of Sigmoid function :

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1-\sigma(x))$$



Pros:

- **Smooth and Differentiable** making it suitable for gradient-based optimization techniques like gradient descent.
- **It is bound to the range (0, 1)** making it useful for probabilistic interpretations.

Activation Functions: Sigmoid

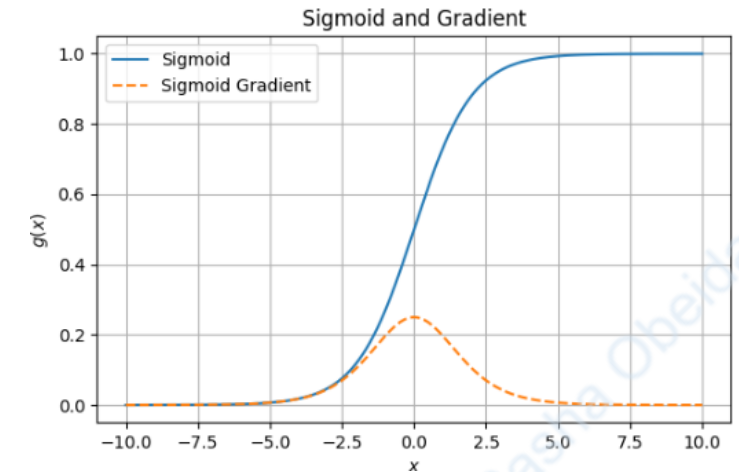
- Sigmoid function : $\sigma(x) = \frac{1}{(1+e^{-x})}$
- Derivative of Sigmoid function :

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1-\sigma(x))$$

Cons

- Computing the exponential function is **expensive**.
- **Saturation of the Function at Extremes:** it has slow convergence (small gradient for large or small input x).
- **Not centered around 0** (has the range (0,1)) which could lead weight swings and slow convergence.
- Has the problem of **vanishing gradients** for very high or very low input values of x .

Vanishing gradients means that gradients tend to get smaller and smaller as we keep on moving backward, which means that earlier layers learn very slowly compared to neurons top layers, eventually , gradient may become zero and learning stops leading to a very bad prediction accuracy.



Activation Functions: Tanh

- Hyperbolic tangent (Tanh):

$$\text{Tanh}(x) = \frac{(1 - e^{-2x})}{(1 + e^{-2x})}$$

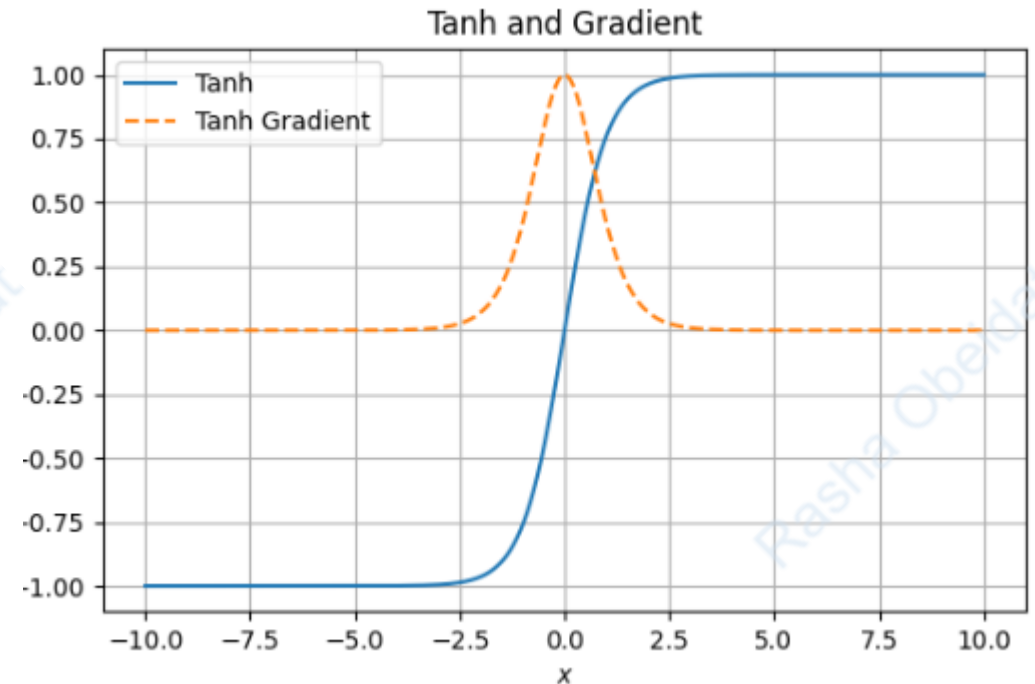
- Derivative of Tanh:

$$\frac{\partial \text{Tanh}(x)}{\partial x} = 1 - (\text{Tanh}(x))^2$$

Pros:

- It is bound to the range (-1, 1).
- The gradient is stronger for tanh than sigmoid , steeper derivatives) makes it converges faster.
- Zero centered, which implies less weight swings(has similar effect of normalization, thus faster convergence.

Commonly to used in Recurrent Neural Networks (RNN).



Activation Functions: Tanh

- Hyperbolic tangent (Tanh):

$$f(x) = \frac{(1 - e^{-2x})}{(1 + e^{-2x})}$$

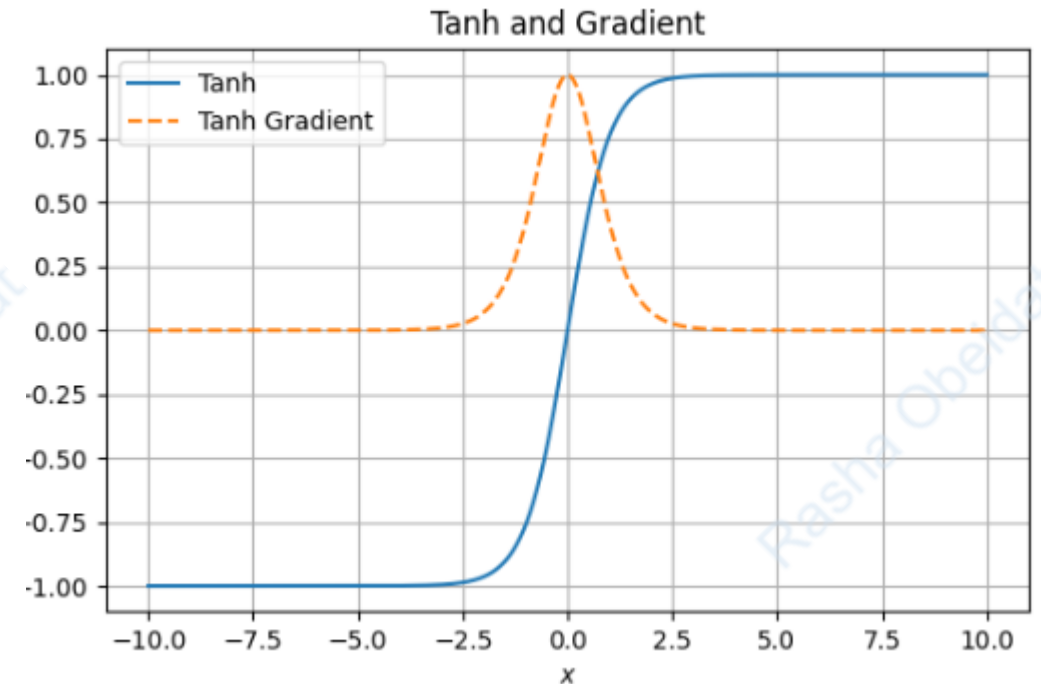
- Derivative of Tanh:

$$\frac{\partial f(x)}{\partial x} = 1 - (\text{Tanh}(x))^2$$

Cons:

Computing the exponential function is expensive.

Has the problem of vanishing gradients for very high or very low values of x .



Activation Functions: ReLU

- Rectified Linear Unit (ReLU):

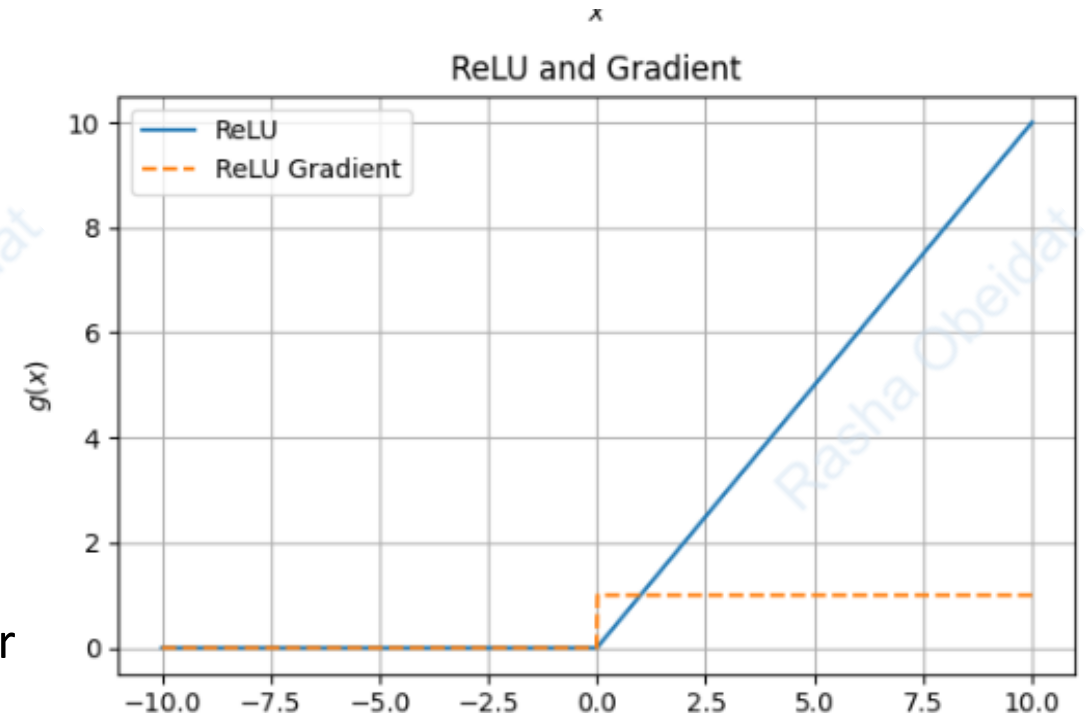
$$f(x) = \max(0, x)$$

- Derivative (sub-gradient) of ReLU :

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1 & , x > 0 \\ 0 \text{ or } 1, & x = 0 \\ 0, & x < 0 \end{cases}$$

Pros:

- Computationally efficient.
- Although it looks like a linear function, it is non-linear because all negative inputs are changed to zero
- Mitigates the vanishing gradient problem.
- Capable of outputting a true zero (sparse representation).
- Avoid small contributions from neurons in early layers in deep networks.
- Converge very fast(7 times faster than Sigmoid)



Activation Functions: ReLU

- Rectified Linear Unit (ReLU):

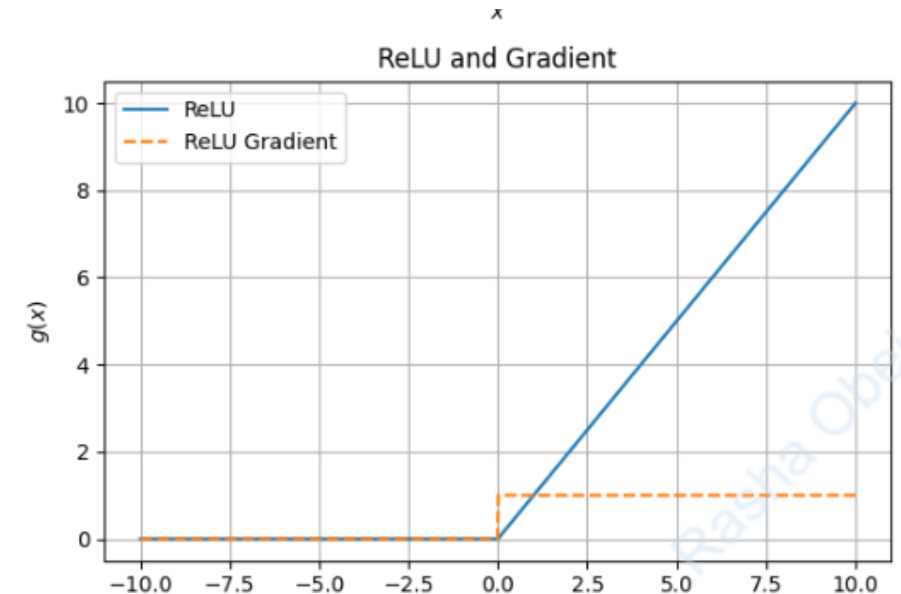
$$f(x) = \max(0, x)$$

- Derivative (sub-gradient) of ReLU :

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1 & , x > 0 \\ 0 \text{ or } 1, & x = 0 \\ 0, & x < 0 \end{cases}$$

Cons:

- **Dying ReLU Problem:** Neurons can become inactive if they consistently output 0 for negative inputs, leading to zero gradients. These "**dead**" neurons no longer contribute to learning, reducing the network's capacity.
- **Unbounded Output:** ReLU outputs can grow indefinitely for large positive inputs, potentially causing instability during training (e.g., **exploding gradients**) in deeper networks without proper regularization or initialization techniques.



Activation Functions: Leaky ReLU

- Leaky Rectified Linear Unit :

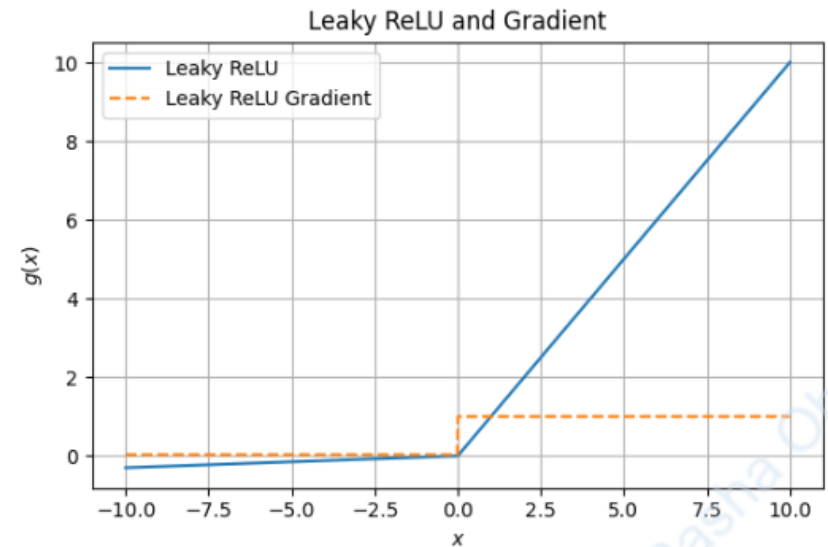
$$f(x) = \max(0.1x, x)$$

- Derivative (sub-gradient) of Leaky ReLU:

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1 & , x > 0 \\ 0.01 \text{ or } 1, & x = 0 \\ 0.01 & , x < 0 \end{cases}$$

Pros:

- Addresses the Dying ReLU Problem:** Unlike ReLU, Leaky ReLU allows a small, non-zero gradient for negative inputs, preventing neurons from becoming inactive and ensuring continuous learning.
- Better Gradient Flow:** The small slope for negative inputs improves gradient flow, particularly in deeper networks, leading to faster and more stable convergence during training.
- Simple Implementation:** Leaky ReLU is computationally efficient and easy to implement, similar to ReLU, without introducing significant complexity.



Activation Functions: Leaky ReLU

- Leaky Rectified Linear Unit :

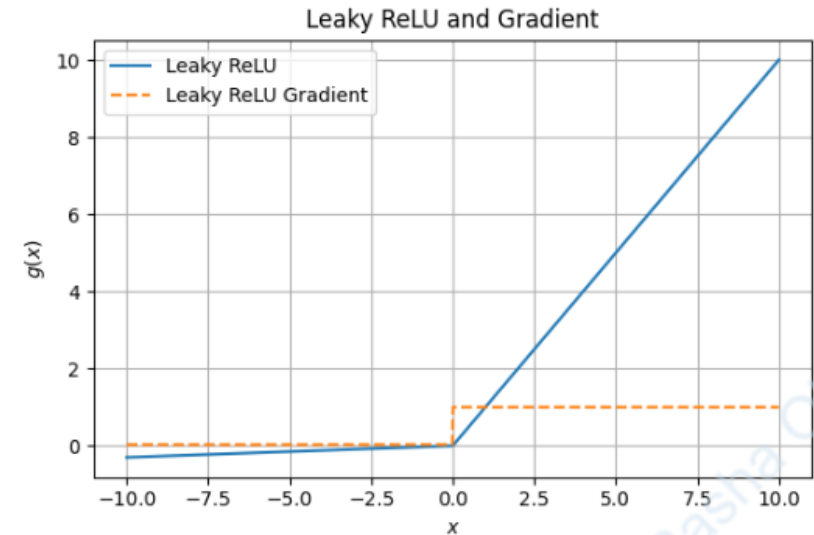
$$f(x) = \max(0.1x, x)$$

- Derivative (sub-gradient) of Leaky ReLU:

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1 & , x > 0 \\ 0.1 \text{ or } 1, & x = 0 \\ 0.1 & , x < 0 \end{cases}$$

Cons:

- Unbounded Output:** Like ReLU, Leaky ReLU outputs can grow indefinitely for large positive inputs, potentially causing instability in deeper networks (e.g., exploding gradients).
- Slope Parameter Selection:** The small negative slope (α) is often chosen arbitrarily (e.g., 0.01), and its optimal value may vary depending on the task or dataset, which can require tuning.
- Not Fully Solving Negative Input Issues:** While Leaky ReLU reduces the problems with negative inputs, it doesn't fully eliminate potential learning challenges caused by small gradients for large negative values.



Parameters vs Hyperparameters

- **Parameters** (or model parameters) are the weight vectors/matrices we learn from the training data the classifier or other machine learning methods. E.g., $W^{[1]}$, $W^{[2]}$ in NN.
- **Hyperparameters** that use to specify the model architecture or use to control training and optimization process, they are usually tuned using hold-out data (validation data). Examples include :
 - Number of hidden nodes.
 - Number of layers,
 - Input features,
 - Learning Rate,
 - Activation Function
 - Regularization terms
 - others

- These slides are the intellectual property of **Dr. Rasha Obeidat®**. If you wish to use them for teaching **your course**, please contact the author for permission.
The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
 - Obeidat.rm@gmail.com
 - Rmobeidat@just.edu.jo

